

LAB ASSIGNMENT 4

NAME - UTKARSH YADAV

ROLL - 23053172

SEC - CSE 33

```
In [55]: import pandas as pd
import numpy as np
import re
import nltk

from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

```
In [56]: nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\KIIT0001\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

```
Out[56]: True
```

```
In [57]: train_df = pd.read_csv("train.csv", encoding="latin-1")
test_df = pd.read_csv("test.csv", encoding="latin-1")

print("Training Data Shape:", train_df.shape)
print("Test Data Shape:", test_df.shape)
```

```
Training Data Shape: (27481, 10)
```

```
Test Data Shape: (4815, 9)
```

```
In [58]: with open("stop-words-list.txt", "r") as file:
stop_words = set(word.strip().lower() for word in file.readlines())

print("Number of stopwords:", len(stop_words))
```

```
Number of stopwords: 127
```

```
In [59]: contraction_map = {
    "can't": "cannot",
    "won't": "will not",
    "n't": " not",
    "'re": " are",
    "'s": " is",
    "'d": " would",
    "'ll": " will",
    "'t": " not",
    "'ve": " have",
    "'m": " am"
}
```

```
def expand_contractions(text):
    for contraction, expanded in contraction_map.items():
        text = re.sub(contraction, expanded, text)
    return text
```

```
In [60]: def preprocess_text(text):
        # Convert to string
        text = str(text).lower()

        # Expand contractions (important!)
        text = expand_contractions(text)

        # Remove special characters
        text = re.sub(r"^[a-zA-Z\s]", "", text)

        # Tokenization
        tokens = word_tokenize(text)

        # Remove stopwords
        tokens = [word for word in tokens if word not in stop_words]

        return tokens
```

```
In [61]: train_df["processed_tokens"] = train_df["text"].apply(preprocess_text)
        test_df["processed_tokens"] = test_df["text"].apply(preprocess_text)

        train_df[["text", "processed_tokens"]].head()
```

```
Out[61]:
```

	text	processed_tokens
0	I'd have responded, if I were going	[id, responded, going]
1	Sooo SAD I will miss you here in San Diego!!!	[soooo, sad, miss, san, diego]
2	my boss is bullying me...	[boss, bullying]
3	what interview! leave me alone	[interview, leave, alone]
4	Sons of ****, why couldn't they put them on t...	[sons, couldnt, put, releases, already, bought]

```
In [62]: train_df["processed_text"] = train_df["processed_tokens"].apply(lambda x: ' '.join(x))
        test_df["processed_text"] = test_df["processed_tokens"].apply(lambda x: ' '.join(x))
```

```
In [63]: unique_words = set()

        for tokens in train_df["processed_tokens"]:
            unique_words.update(tokens)

        print("Total Unique Words:", len(unique_words))
        list(unique_words)[:30]
```

Total Unique Words: 27884

```
Out[63]: ['hacker',
          'chickadee',
          'degrees',
          'blender',
          'ams',
          'looooooooooooooooool',
          'passion',
          'hauer',
          'sleep',
          'attention',
          'koozie',
          'marie',
          'nowor',
          'titlewhoops',
          'fink',
          'adoreeeee',
          'fused',
          'httpbitlyfgly',
          'valentinos',
          'grandad',
          'lorraines',
          'iq',
          'httptinyurlcomcfre',
          'isoulja',
          'sighhh',
          'backlog',
          'huhuhu',
          'luca',
          'ouchies',
          'httpblipfmqvz']
```

```
In [64]: tfidf = TfidfVectorizer()

X_train = tfidf.fit_transform(train_df["processed_text"])
X_test = tfidf.transform(test_df["processed_text"])

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

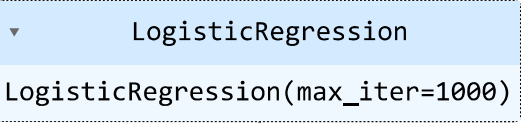
```
X_train shape: (27481, 27862)
X_test shape: (4815, 27862)
```

```
In [65]: y_train = train_df["sentiment"]
y_test = test_df["sentiment"]
```

```
In [66]: # Saving feature matrices and labels
pd.DataFrame(X_train.toarray()).to_csv("X_train.csv", index=False)
pd.DataFrame(X_test.toarray()).to_csv("X_test.csv", index=False)

y_train.to_csv("y_train.csv", index=False)
y_test.to_csv("y_test.csv", index=False)
```

```
In [67]: model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

Out[67]: 

```
In [68]: y_pred = model.predict(X_test)
```

```
In [69]: print("y_test type:", type(y_test))
print("y_pred type:", type(y_pred))

print("\ny_test unique values:")
print(y_test.unique())

print("\nAny NaN in y_test?")
print(y_test.isna().sum())

print("\nLength check:")
print(len(y_test), len(y_pred))
```

```
y_test type: <class 'pandas.core.series.Series'>
y_pred type: <class 'numpy.ndarray'>
```

```
y_test unique values:
['neutral' 'positive' 'negative' nan]
```

```
Any NaN in y_test?
1281
```

```
Length check:
4815 4815
```

```
In [70]: # Combine y_test and predictions to align indices
results_df = test_df.copy()
results_df["predicted_sentiment"] = y_pred

# Drop rows where true sentiment is missing
results_df = results_df.dropna(subset=["sentiment"])

y_test_clean = results_df["sentiment"]
y_pred_clean = results_df["predicted_sentiment"]

accuracy = accuracy_score(y_test_clean, y_pred_clean)
print("Accuracy:", accuracy)

print("\nClassification Report:\n")
print(classification_report(y_test_clean, y_pred_clean))
```

Accuracy: 0.6992076966610073

Classification Report:

	precision	recall	f1-score	support
negative	0.72	0.63	0.67	1001
neutral	0.63	0.75	0.68	1430
positive	0.80	0.71	0.75	1103
accuracy			0.70	3534
macro avg	0.72	0.69	0.70	3534
weighted avg	0.71	0.70	0.70	3534

```
In [71]: test_df["predicted_sentiment"] = y_pred
test_df[["text", "sentiment", "predicted_sentiment"]].head()
```

```
Out[71]:
```

	text	sentiment	predicted_sentiment
0	Last session of the day http://twitpic.com/67ezh	neutral	neutral
1	Shanghai is also really exciting (precisely -...	positive	positive
2	Recession hit Veronique Branquinho, she has to...	negative	negative
3	happy bday!	positive	positive
4	http://twitpic.com/4w75p - I like it!!	positive	positive